

# Sistema de Gestão de Territórios e Publicações (GTP)

## Documento 18 – Guia de Instalação e Configuração

---

**Projeto:** Sistema de Gestão de Territórios e Publicações (GTP)

**Versão:** 1.0.0

**Status:** Em Elaboração

**Data:** Julho/2026

**Autor:** Fabio André Zatta

---

# 1 – Introdução

## 1.1 Objetivo

Este documento tem como objetivo descrever todos os procedimentos necessários para instalar, configurar e colocar em funcionamento o **Gestor de Territórios e Publicações (GTP)** em diferentes ambientes.

Ao final deste guia será possível instalar completamente a plataforma utilizando as tecnologias adotadas pelo projeto, garantindo um ambiente padronizado, seguro e reproduzível.

Este documento complementa:

- Documento 05 – Arquitetura Geral
- Documento 06 – Arquitetura Backend
- Documento 07 – Arquitetura Frontend
- Documento 10 – API REST
- Documento 12 – Banco de Dados PostgreSQL
- Documento 14 – Deploy e Infraestrutura
- Documento 15 – Manual do Desenvolvedor

## 1.2 Escopo

Este guia contempla a instalação completa dos seguintes componentes:

- Backend Spring Boot
- Frontend React
- PostgreSQL
- Flyway
- Docker
- Docker Compose
- Java JDK 21
- Maven
- Node.js
- Vite
- Nginx (quando utilizado)
- Certificados HTTPS
- Variáveis de Ambiente
- Ambiente Local
- Ambiente de Homologação
- Ambiente de Produção

## 1.3 Público-Alvo

Este documento destina-se a:

- Desenvolvedores
- Administradores do Sistema
- DevOps
- Administradores de Banco de Dados
- Equipe de Infraestrutura
- Equipe de Suporte

## 1.4 Tecnologias Utilizadas

A plataforma GTP utiliza a seguinte arquitetura tecnológica.

| <b>Camada</b>  | <b>Tecnologia</b>           |
|----------------|-----------------------------|
| Frontend       | React 19 + Vite             |
| Backend        | Java 21 + Spring Boot 3     |
| Banco          | PostgreSQL 17               |
| ORM            | Spring Data JPA + Hibernate |
| Migração       | Flyway                      |
| Segurança      | Spring Security + JWT       |
| Build Backend  | Maven                       |
| Build Frontend | npm / Vite                  |
| Contêiner      | Docker                      |
| Orquestração   | Docker Compose              |
| Proxy Reverso  | Nginx                       |
| Versionamento  | Git + GitHub                |

## 1.5 Ambientes Suportados

O GTP foi projetado para funcionar em diferentes ambientes.

| <b>Ambiente</b> | <b>Finalidade</b>         |
|-----------------|---------------------------|
| Desenvolvimento | Construção do sistema     |
| Homologação     | Testes integrados         |
| Produção        | Utilização pelos usuários |

Cada ambiente possui configurações próprias.

## 1.6 Pré-Requisitos

Antes da instalação é necessário possuir:

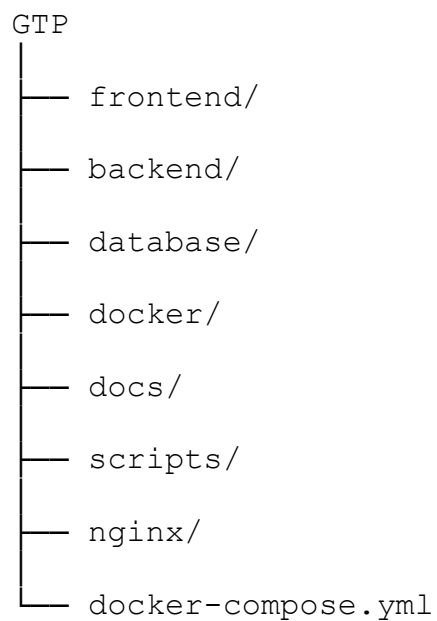
### Sistema Operacional

- Windows 11
- Ubuntu 24 LTS
- Debian 12
- Fedora
- macOS

### Software

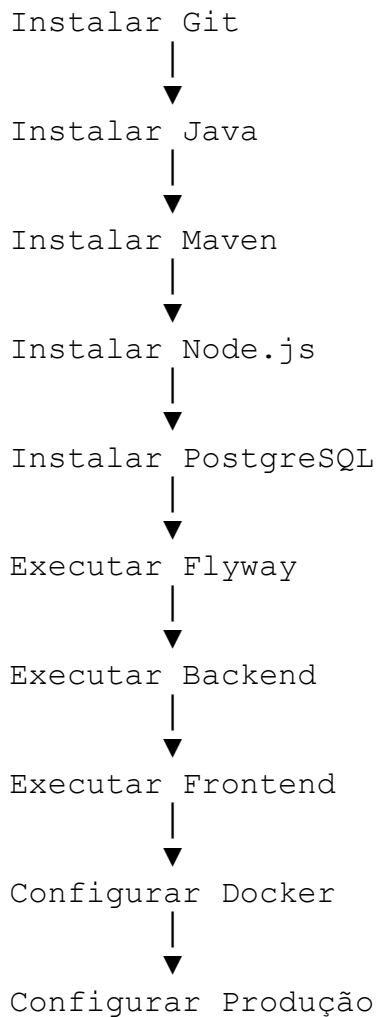
- Git
- Java JDK 21
- Maven 3.9+
- Node.js LTS
- npm
- PostgreSQL 17
- Docker
- Docker Compose

## 1.7 Estrutura Geral da Aplicação



## 1.8 Processo Geral de Instalação

O processo de instalação seguirá a seguinte sequência:



## 1.9 Convenções

Neste documento serão utilizadas as seguintes convenções:

- comandos executados no terminal;
- exemplos completos de arquivos;
- estruturas de diretórios;
- boas práticas de configuração;
- recomendações de segurança.

Todos os exemplos apresentados deverão ser adaptados conforme o ambiente de instalação.

# 1.10 Conclusão do Capítulo

Este capítulo apresentou os objetivos, o escopo e a organização do **Guia de Instalação e Configuração**, estabelecendo os pré-requisitos e a visão geral do processo de implantação do GTP.

Nos próximos capítulos serão detalhados os procedimentos necessários para preparar o ambiente de desenvolvimento, configurar os componentes da aplicação e realizar a implantação em diferentes ambientes de execução.

## 2 – Instalação do Ambiente de Desenvolvimento

### 2.1 Objetivo

Este capítulo descreve como preparar um ambiente de desenvolvimento completo para o **Gestor de Territórios e Publicações (GTP)**.

Ao final deste procedimento, o desenvolvedor terá todas as ferramentas necessárias instaladas e configuradas para executar o backend, o frontend e o banco de dados localmente.

O ambiente descrito neste documento é compatível com a arquitetura definida nos documentos técnicos do projeto e busca garantir uma configuração padronizada entre todos os membros da equipe.

### 2.2 Requisitos de Hardware

Embora o GTP possa ser executado em diferentes configurações de hardware, recomenda-se a seguinte especificação mínima para desenvolvimento.

| Recurso                | Recomendado                                                          |
|------------------------|----------------------------------------------------------------------|
| Processador            | 4 núcleos (Intel Core i5, AMD Ryzen 5 ou superior)                   |
| Memória RAM            | 16 GB                                                                |
| Armazenamento          | SSD com pelo menos 30 GB livres                                      |
| Resolução de Tela      | 1920 × 1080 ou superior                                              |
| Conexão com a Internet | Necessária para instalação de dependências e acesso aos repositórios |

Para utilização simultânea de Docker, IDE e banco de dados, recomenda-se 32 GB de memória RAM.

## 2.3 Sistemas Operacionais Suportados

O ambiente pode ser configurado nos seguintes sistemas operacionais:

**Sistema    Versão Recomendada**

Windows    Windows 11

Ubuntu    24.04 LTS

Debian    12 ou superior

Fedora    Última versão estável

macOS    Última versão compatível com Java 21

Os exemplos deste documento utilizam comandos para Windows PowerShell e Linux (Bash).

## 2.4 Ferramentas Necessárias

Antes de iniciar o desenvolvimento, instale os seguintes componentes.

| Ferramenta                     | Versão Recomendada                 |
|--------------------------------|------------------------------------|
| Git                            | 2.45 ou superior                   |
| Java JDK                       | 21 LTS                             |
| Maven                          | 3.9 ou superior                    |
| Node.js                        | LTS 22.x ou superior               |
| npm                            | Compatível com a versão do Node.js |
| PostgreSQL                     | 17                                 |
| Docker Desktop / Docker Engine | Última versão estável              |
| Docker Compose                 | Versão integrada ao Docker         |

## Ferramenta

## Versão Recomendada

Visual Studio Code

Última versão estável

## 2.5 Instalação do Git

O Git é utilizado para controle de versão e integração com o GitHub.

Após a instalação, verifique a versão:

### Windows / Linux

```
git --version
```

Resultado esperado:

```
git version 2.x.x
```

Configure sua identidade:

```
git config --global user.name "Seu Nome"
```

```
git config --global user.email "seu@email.com"
```

## 2.6 Instalação do Java JDK 21

O backend do GTP utiliza **Java 21 LTS**.

Após a instalação:

```
java -version
```

Resultado esperado:

```
openjdk version "21"
```

Verifique também:

```
javac -version
```

## 2.7 Configuração da Variável JAVA\_HOME

Configure a variável de ambiente **JAVA\_HOME** apontando para o diretório de instalação do JDK.

Exemplo:



## Windows

```
JAVA_HOME=C:\Program Files\Java\jdk-21
```

## Linux

```
export JAVA_HOME=/usr/lib/jvm/jdk-21
```

Inclua também o diretório bin do Java na variável PATH.

## 2.8 Instalação do Maven

Após instalar o Maven, verifique a instalação:

```
mvn -version
```

Resultado esperado:

```
Apache Maven 3.9.x
```

```
Java version: 21
```

O Maven será responsável pelo gerenciamento das dependências e pela construção do backend.

## 2.9 Instalação do Node.js

O frontend do GTP utiliza React com Vite.

Verifique a instalação:

```
node -v
```

```
npm -v
```

Resultado esperado:

```
Node.js v22.x.x
```

```
npm 10.x.x
```

## 2.10 Instalação do PostgreSQL

Durante a instalação do PostgreSQL, recomenda-se configurar:

| Configuração                   | Valor                                     |
|--------------------------------|-------------------------------------------|
| Porta                          | 5432                                      |
| Codificação                    | UTF-8                                     |
| Locale                         | pt_BR.UTF-8 (quando aplicável)            |
| Usuário administrador postgres |                                           |
| Senha                          | Definida conforme política da organização |

Após a instalação, verifique o serviço:

```
psql --version
```

## 2.11 Instalação do Docker

O Docker será utilizado para execução isolada dos serviços do GTP.

Verifique a instalação:

```
docker --version
```

E confirme que o Docker Compose está disponível:

```
docker compose version
```

Resultado esperado:

```
Docker version XX
```

```
Docker Compose version XX
```

## 2.12 Instalação do Visual Studio Code

O Visual Studio Code é a IDE recomendada para o desenvolvimento do projeto.

Extensões sugeridas:

- Extension Pack for Java
- Spring Boot Extension Pack
- ESLint
- Prettier
- Docker

- GitLens
- PostgreSQL
- EditorConfig
- IntelliCode

Essas extensões aumentam a produtividade e padronizam o ambiente de desenvolvimento.

## 2.13 Clonagem do Repositório

Clone o repositório do projeto:

```
git clone
https://github.com/fisiofaz/SuperitendenteDeServico-
Gestao.git
```

Acesse a pasta do projeto:

```
cd SuperitendenteDeServico-Gestao
```

## 2.14 Estrutura Esperada do Projeto

Após a clonagem, a estrutura deverá ser semelhante a:

```
SuperitendenteDeServico-Gestao
|
├── backend
├── frontend
├── docs
├── docker
├── nginx
├── scripts
├── database
├── .gitignore
├── docker-compose.yml
└── README.md
```

Essa organização separa claramente os componentes da aplicação.

## 2.15 Verificação do Ambiente

Antes de prosseguir, confirme que todas as ferramentas estão corretamente instaladas.

| Comando                             | Resultado Esperado        |
|-------------------------------------|---------------------------|
| <code>git --version</code>          | Versão do Git             |
| <code>java -version</code>          | Java 21                   |
| <code>mvn -version</code>           | Maven 3.9+                |
| <code>node -v</code>                | Node.js LTS               |
| <code>npm -v</code>                 | npm compatível            |
| <code>psql --version</code>         | PostgreSQL 17             |
| <code>docker --version</code>       | Docker instalado          |
| <code>docker compose version</code> | Docker Compose disponível |

Caso algum comando apresente erro, recomenda-se corrigir a instalação antes de continuar.

## 2.16 Boas Práticas para o Ambiente

Para manter um ambiente de desenvolvimento consistente, recomenda-se:

- utilizar versões padronizadas das ferramentas;
- manter o sistema operacional atualizado;
- utilizar um gerenciador de senhas para credenciais locais;
- evitar alterações manuais em dependências gerenciadas;
- sincronizar frequentemente o repositório com a branch principal;
- utilizar ambientes isolados para testes experimentais.

## 2.17 Problemas Frequentes

| Problema                     | Possível Solução                                        |
|------------------------------|---------------------------------------------------------|
| java não encontrado          | Verificar a configuração do JAVA_HOME e do PATH         |
| mvn não reconhecido          | Confirmar a instalação do Maven e a variável PATH       |
| node não encontrado          | Reinstalar o Node.js e verificar a variável PATH        |
| PostgreSQL não inicia        | Verificar o serviço e a porta configurada               |
| Docker não inicia            | Confirmar se o serviço do Docker está em execução       |
| Erro ao clonar o repositório | Validar a URL do repositório e as credenciais do GitHub |

## 2.18 Conclusão do Capítulo

Este capítulo apresentou os procedimentos necessários para preparar um ambiente de desenvolvimento completo para o GTP, incluindo a instalação das ferramentas, a configuração das variáveis de ambiente e a clonagem do repositório do projeto.

Com o ambiente devidamente configurado, o desenvolvedor estará apto a instalar e executar os componentes da aplicação.

Esse capítulo detalhará a preparação do ambiente backend do **Gestor de Territórios e Publicações (GTP)**, permitindo a execução da aplicação de forma integrada com o banco de dados e os demais componentes da arquitetura.

## 3 – Configuração do Backend

### 3.1 Objetivo

Este capítulo descreve a configuração do **backend** do **Gestor de Territórios e Publicações (GTP)**, desenvolvido com **Java 21**, **Spring Boot 3** e **PostgreSQL**.

Serão apresentados os procedimentos necessários para configurar o ambiente, ajustar os arquivos de configuração, conectar ao banco de dados e executar a aplicação localmente.

## 3.2 Visão Geral da Arquitetura

O backend do GTP é responsável por:

- disponibilizar a API REST;
- implementar as regras de negócio;
- controlar autenticação e autorização;
- realizar persistência dos dados;
- executar migrações do banco de dados;
- integrar os módulos da plataforma.

A arquitetura segue o padrão em camadas, promovendo organização, reutilização e facilidade de manutenção.

## 3.3 Pré-Requisitos

Antes da configuração, confirme que os seguintes componentes estão instalados:

### Componente Versão Recomendada

|            |                  |
|------------|------------------|
| Java       | 21 LTS           |
| Maven      | 3.9 ou superior  |
| PostgreSQL | 17               |
| Git        | 2.45 ou superior |

Além disso, o banco de dados deve estar em execução e acessível.

## 3.4 Estrutura do Projeto Backend

A estrutura principal do backend segue a organização abaixo:

```

backend/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── br/
│   │   │   │   ├── com/
│   │   │   │   │   ├── gtp/
│   │   │   │   │   │   ├── config/
│   │   │   │   │   │   ├── controller/
│   │   │   │   │   │   ├── dto/
│   │   │   │   │   │   ├── entity/
│   │   │   │   │   │   ├── exception/
│   │   │   │   │   │   ├── mapper/
│   │   │   │   │   │   ├── repository/
│   │   │   │   │   │   ├── security/
│   │   │   │   │   │   ├── service/
│   │   │   │   │   │   └── util/
│   │   │   └── resources/
│   │   │       ├── db/
│   │   │       │   └── migration/
│   │   │       ├── application.yml
│   │   │       └── application-dev.yml
│   └── test/
├── pom.xml
└── mvnw

```

Essa estrutura facilita a separação das responsabilidades e segue as boas práticas recomendadas para projetos Spring Boot.

### 3.5 Configuração do Banco de Dados

Antes da primeira execução, crie o banco de dados PostgreSQL conforme definido no projeto.

Exemplo:

| Configuração   | Valor                     |
|----------------|---------------------------|
| Banco de Dados | gtp_db                    |
| Usuário        | gtp_user                  |
| Senha          | Definida pela organização |

| Configuração | Valor |
|--------------|-------|
|--------------|-------|

|       |      |
|-------|------|
| Porta | 5432 |
|-------|------|

Os valores devem ser ajustados conforme o ambiente de desenvolvimento.

## 3.6 Configuração do Arquivo application.yml

O arquivo application.yml concentra as configurações gerais da aplicação.

Entre os principais grupos de configuração estão:

- identificação da aplicação;
- conexão com o banco de dados;
- JPA/Hibernate;
- Flyway;
- autenticação JWT;
- logs;
- CORS;
- perfis de execução.

Recomenda-se manter informações sensíveis em variáveis de ambiente e evitar o armazenamento de credenciais diretamente no repositório.

## 3.7 Perfis de Execução

O Spring Boot permite utilizar diferentes perfis para cada ambiente.

Os perfis recomendados são:

| Perfil | Finalidade            |
|--------|-----------------------|
| dev    | Desenvolvimento local |
| test   | Testes automatizados  |
| hom    | Homologação           |
| prod   | Produção              |



Cada perfil possui configurações específicas, permitindo adaptar a aplicação sem alterar o código-fonte.

## 3.8 Configuração do Flyway

O Flyway é responsável pelo versionamento do banco de dados.

As migrações devem ser armazenadas no diretório:

`src/main/resources/db/migration`

Boas práticas:

- uma migração por alteração estrutural;
- não editar migrações já aplicadas;
- utilizar nomenclatura sequencial;
- manter as migrações sob controle de versão.

Durante a inicialização da aplicação, o Flyway executará automaticamente as migrações pendentes.

## 3.9 Compilação do Projeto

Antes da execução, recomenda-se compilar o projeto para verificar possíveis inconsistências.

Na pasta do backend, execute:

`mvn clean install`

Esse processo realiza:

- limpeza de artefatos anteriores;
- download de dependências;
- compilação do código;
- execução dos testes configurados;
- geração do pacote da aplicação.

## 3.10 Execução da Aplicação

Após a compilação, inicie o backend:

```
mvn spring-boot:run
```

Caso tudo esteja configurado corretamente, a aplicação iniciará e ficará disponível na porta configurada (por padrão, 8080).

## 3.11 Validação da Inicialização

Após iniciar a aplicação, verifique:

- conexão com o PostgreSQL;
- execução das migrações Flyway;
- inicialização do Spring Boot;
- carregamento dos componentes;
- ausência de erros críticos nos logs.

A API poderá ser testada utilizando ferramentas como Postman, Insomnia ou Swagger (quando habilitado).

## 3.12 Variáveis de Ambiente

Informações sensíveis não devem ser armazenadas diretamente nos arquivos de configuração.

Recomenda-se utilizar variáveis de ambiente para:

- URL do banco de dados;
- usuário e senha do PostgreSQL;
- chave secreta do JWT;
- credenciais SMTP;
- parâmetros de integração com serviços externos.

Essa prática aumenta a segurança e facilita a configuração em diferentes ambientes.

### 3.13 Configuração de Logs

O backend utiliza o mecanismo de logs do Spring Boot para registrar eventos da aplicação.

Os níveis recomendados são:

| Nível | Utilização                                 |
|-------|--------------------------------------------|
| INFO  | Operação normal da aplicação               |
| WARN  | Situações que exigem atenção               |
| ERROR | Falhas que impedem a execução de operações |
| DEBUG | Diagnóstico em ambiente de desenvolvimento |

Em produção, recomenda-se evitar o uso contínuo do nível DEBUG, pois pode impactar o desempenho e expor informações desnecessárias.

### 3.14 Verificação da API

Após a inicialização, recomenda-se validar:

- autenticação de usuários;
- endpoints públicos;
- endpoints protegidos;
- operações CRUD;
- respostas HTTP;
- tratamento de erros;
- integração com o banco de dados.

Esses testes garantem que o backend esteja funcionando corretamente.

### 3.15 Problemas Frequentes

| Problema                    | Possível Solução                                              |
|-----------------------------|---------------------------------------------------------------|
| Erro de conexão com o banco | Verificar URL, usuário, senha e disponibilidade do PostgreSQL |

| <b>Problema</b>              | <b>Possível Solução</b>                                                        |
|------------------------------|--------------------------------------------------------------------------------|
| Flyway não executa           | Confirmar localização das migrações e configuração do Flyway                   |
| Porta já utilizada           | Alterar a porta da aplicação ou encerrar o processo conflitante                |
| Dependências não encontradas | Executar novamente mvn clean install e verificar acesso aos repositórios Maven |
| Erro de autenticação         | Revisar configurações de segurança e credenciais utilizadas                    |
| Aplicação não inicia         | Analisar os logs para identificar a causa da falha                             |

### 3.16 Boas Práticas

Durante o desenvolvimento do backend, recomenda-se:

- utilizar perfis específicos para cada ambiente;
- manter credenciais em variáveis de ambiente;
- executar migrações apenas por meio do Flyway;
- documentar alterações estruturais no banco de dados;
- revisar os logs após cada implantação;
- manter as dependências atualizadas;
- realizar testes antes de enviar alterações ao repositório principal.

### 3.17 Conclusão do Capítulo

Este capítulo apresentou os procedimentos necessários para configurar o backend do GTP, abordando a estrutura do projeto, a conexão com o banco de dados, a utilização do Flyway, os perfis de execução, a compilação e a inicialização da aplicação.

Com essa configuração concluída, o backend estará preparado para fornecer os serviços da API REST e integrar-se aos demais componentes da plataforma.

## 4 – Configuração do Frontend

### 4.1 Objetivo

Este capítulo descreve os procedimentos para configurar o **frontend** do **Gestor de Territórios e Publicações (GTP)**, desenvolvido com **React 19**, **Vite** e **React Router**.

Serão apresentados os passos necessários para preparar o ambiente, instalar as dependências, configurar as variáveis de ambiente, integrar a interface com a API REST e executar a aplicação localmente.

### 4.2 Visão Geral da Arquitetura

O frontend do GTP é responsável por fornecer uma interface moderna, responsiva e intuitiva para os usuários da plataforma.

Suas principais responsabilidades incluem:

- autenticação de usuários;
- navegação entre módulos;
- consumo da API REST;
- validação de formulários;
- gerenciamento do estado da aplicação;
- exibição de relatórios e dashboards;
- administração de usuários, territórios e publicações.

A aplicação foi desenvolvida utilizando uma arquitetura baseada em componentes reutilizáveis e organização por funcionalidades (*feature-based architecture*).

### 4.3 Pré-Requisitos

Antes da configuração do frontend, confirme que os seguintes componentes estão instalados:

| Componente | Versão Recomendada       |
|------------|--------------------------|
| Node.js    | 22 LTS ou superior       |
| npm        | Compatível com o Node.js |
| Git        | 2.45 ou superior         |

Visual Studio Code Última versão estável

Além disso, o backend do GTP deve estar em execução para que a interface possa consumir os serviços da API.

## 4.4 Estrutura do Projeto Frontend

A estrutura principal do frontend segue uma organização modular por funcionalidades.

```
frontend/
├── public/
├── src/
│   ├── assets/
│   ├── components/
│   ├── contexts/
│   ├── features/
│   │   ├── dashboard/
│   │   ├── usuarios/
│   │   ├── publicadores/
│   │   ├── territorios/
│   │   ├── publicacoes/
│   │   ├── relatorios/
│   │   └── configuracoes/
│   ├── hooks/
│   ├── layouts/
│   ├── pages/
│   ├── routes/
│   ├── services/
│   ├── styles/
│   ├── utils/
│   ├── App.jsx
│   └── main.jsx
├── package.json
├── vite.config.js
└── .env
```

Essa estrutura favorece a escalabilidade e facilita a manutenção da aplicação.

## 4.5 Instalação das Dependências

Após acessar o diretório do frontend, instale as dependências do projeto:

```
npm install
```

Esse comando instalará automaticamente todas as bibliotecas definidas no arquivo `package.json`.

## 4.6 Configuração das Variáveis de Ambiente

O frontend utiliza variáveis de ambiente para configurar a comunicação com o backend e outros serviços.

Recomenda-se criar um arquivo `.env` na raiz do projeto contendo apenas informações específicas do ambiente, como:

- URL da API;
- nome da aplicação;
- ambiente de execução;
- parâmetros de funcionalidades opcionais (*feature flags*), quando utilizados.

As variáveis devem seguir o padrão exigido pelo Vite, iniciando com o prefixo `VITE_`.

Informações sensíveis, como senhas ou chaves privadas, **não devem ser armazenadas no frontend**, pois o código é distribuído ao navegador do usuário.

## 4.7 Configuração da Comunicação com a API

A comunicação com o backend é realizada por meio de um cliente HTTP centralizado.

Recomenda-se que toda requisição utilize:

- URL base definida por variável de ambiente;
- envio automático do token JWT nas requisições autenticadas;
- tratamento centralizado de erros;

- renovação de autenticação, quando aplicável;
- interceptadores para registro e monitoramento de falhas.

Essa abordagem reduz duplicação de código e facilita futuras alterações.

## 4.8 Execução da Aplicação

Após instalar as dependências e configurar o ambiente, inicie o servidor de desenvolvimento:

```
npm run dev
```

Por padrão, o Vite disponibilizará a aplicação em uma porta local (geralmente 5173).

Ao acessar o endereço exibido no terminal, a interface do GTP estará disponível para utilização.

## 4.9 Build para Produção

Para gerar a versão otimizada da aplicação, execute:

```
npm run build
```

O processo realiza:

- compilação dos componentes;
- otimização dos arquivos JavaScript;
- minimização de CSS;
- geração dos arquivos estáticos;
- preparação para publicação em ambiente de produção.

Os arquivos gerados serão armazenados no diretório dist/.

## 4.10 Validação da Interface

Após iniciar a aplicação, recomenda-se validar:

- tela de login;
- autenticação;



- navegação entre páginas;
- carregamento dos menus;
- funcionamento dos formulários;
- comunicação com a API;
- exibição de mensagens de erro;
- funcionamento das permissões de acesso.

Esses testes confirmam a integração correta entre frontend e backend.

## 4.11 Organização dos Componentes

Os componentes do GTP seguem os princípios de reutilização e responsabilidade única.

Boas práticas adotadas:

- componentes pequenos e reutilizáveis;
- separação entre lógica e apresentação;
- uso de componentes compartilhados para elementos comuns;
- organização por funcionalidades;
- padronização de nomenclaturas.

Essa abordagem facilita a evolução da interface.

## 4.12 Gerenciamento de Estado

O gerenciamento do estado da aplicação é realizado por meio de Context API e hooks personalizados, podendo ser expandido futuramente para outras soluções, caso necessário.

Os principais estados gerenciados incluem:

- autenticação do usuário;
- dados da sessão;
- permissões de acesso;
- preferências da interface;

- informações compartilhadas entre componentes.

A centralização do estado reduz inconsistências e simplifica a manutenção.

## 4.13 Configuração de Rotas

O GTP utiliza um sistema de roteamento para organizar a navegação entre os módulos.

As rotas são divididas em:

| <b>Tipo</b>     | <b>Finalidade</b>                            |
|-----------------|----------------------------------------------|
| Públicas        | Acessíveis sem autenticação                  |
| Protegidas      | Exigem autenticação e permissões específicas |
| Administrativas | Restritas aos perfis autorizados             |

O controle de acesso é realizado por componentes responsáveis por validar a autenticação e as permissões antes de permitir a navegação.

## 4.14 Problemas Frequentes

| <b>Problema</b>                | <b>Possível Solução</b>                                                                       |
|--------------------------------|-----------------------------------------------------------------------------------------------|
| Erro ao executar npm install   | Verificar conexão com a internet e versão do Node.js                                          |
| Aplicação não inicia           | Confirmar se todas as dependências foram instaladas                                           |
| Falha na comunicação com a API | Revisar a URL configurada nas variáveis de ambiente e verificar se o backend está em execução |
| Tela em branco                 | Consultar o console do navegador para identificar erros de JavaScript                         |
| Erros de CORS                  | Validar a configuração de CORS no backend                                                     |
| Build com falha                | Revisar dependências, imports e mensagens apresentadas durante a compilação                   |

## 4.15 Boas Práticas

Durante o desenvolvimento do frontend, recomenda-se:

- manter as dependências atualizadas;
- utilizar componentes reutilizáveis;
- evitar lógica complexa diretamente na interface;
- documentar componentes compartilhados;
- utilizar variáveis de ambiente para configurações;
- testar a interface em diferentes resoluções;
- validar a responsividade antes de cada entrega.

## 4.16 Conclusão do Capítulo

Este capítulo apresentou os procedimentos necessários para configurar o frontend do GTP, incluindo a instalação das dependências, a organização da estrutura do projeto, a configuração das variáveis de ambiente, a integração com a API REST e a geração da versão de produção.

Com essa configuração concluída, a interface estará pronta para consumir os serviços disponibilizados pelo backend e oferecer aos usuários uma experiência consistente, segura e responsiva.

## 5 – Configuração do PostgreSQL e Flyway

### 5.1 Objetivo

Este capítulo apresenta os procedimentos para instalação, configuração e integração do **PostgreSQL** com o **Flyway**, utilizados como solução de persistência de dados do **Gestor de Territórios e Publicações (GTP)**.

Serão abordadas as etapas necessárias para criar o banco de dados, configurar usuários e permissões, executar migrações e validar a estrutura da base de dados.

## 5.2 Visão Geral

O GTP utiliza o **PostgreSQL 17** como Sistema Gerenciador de Banco de Dados (SGBD) e o **Flyway** para controle de versão e evolução do esquema do banco.

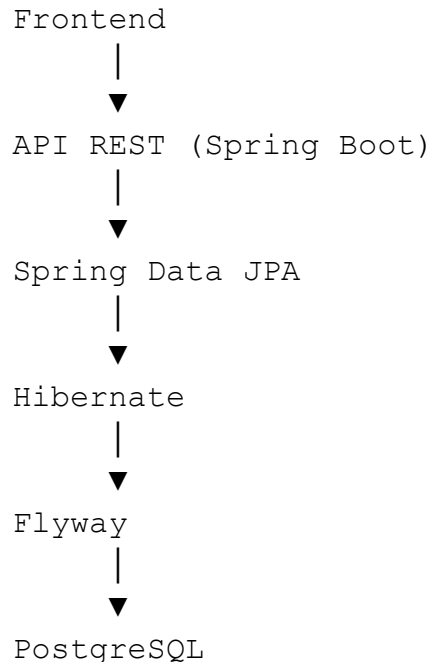
Essa combinação oferece:

- versionamento das alterações estruturais;
- rastreabilidade das migrações;
- consistência entre os ambientes;
- facilidade para implantação contínua;
- suporte a rollback lógico por meio de novas migrações corretivas.

O banco de dados representa a fonte oficial de armazenamento das informações da plataforma.

## 5.3 Arquitetura da Persistência

A arquitetura de persistência é composta pelos seguintes elementos:



Cada camada possui responsabilidades específicas, garantindo desacoplamento e facilidade de manutenção.

## 5.4 Instalação do PostgreSQL

Instale o PostgreSQL utilizando a versão recomendada para o projeto.

Configurações sugeridas durante a instalação:

| Configuração          | Valor Recomendado               |
|-----------------------|---------------------------------|
| Versão                | PostgreSQL 17                   |
| Porta                 | 5432                            |
| Codificação           | UTF-8                           |
| Locale                | pt_BR.UTF-8 (quando disponível) |
| Usuário administrador | postgres                        |
| Serviço               | Inicialização automática        |

Após a instalação, confirme que o serviço está em execução.

## 5.5 Criação do Banco de Dados

Crie um banco de dados exclusivo para o GTP.

Configuração sugerida:

| Item         | Valor    |
|--------------|----------|
| Banco        | gtp_db   |
| Codificação  | UTF-8    |
| Proprietário | gtp_user |

O nome do banco poderá ser alterado conforme a política da organização.

## 5.6 Criação do Usuário da Aplicação

Recomenda-se utilizar um usuário específico para a aplicação, evitando o uso da conta administrativa do PostgreSQL.

Exemplo de configuração:

## **Configuração Recomendação**

Usuário        gtp\_user

Senha         Definida conforme política de segurança

Permissões    Apenas as necessárias para a aplicação

Essa abordagem segue o princípio do menor privilégio.

## **5.7 Configuração da Conexão**

O backend deverá ser configurado para acessar o banco de dados utilizando os parâmetros definidos para o ambiente.

As informações de conexão incluem:

- endereço do servidor;
- porta;
- nome do banco;
- usuário;
- senha;
- parâmetros de conexão.

Esses dados devem ser fornecidos por meio de variáveis de ambiente ou arquivos de configuração específicos para cada ambiente.

## **5.8 Configuração do Flyway**

O Flyway é responsável por controlar todas as alterações estruturais do banco de dados.

Durante a inicialização do backend, o Flyway:

1. identifica a versão atual do banco;
2. verifica as migrações pendentes;
3. executa os scripts necessários;
4. registra as alterações realizadas;
5. atualiza o histórico de migrações.

Esse processo garante que todos os ambientes permaneçam sincronizados.

## 5.9 Estrutura das Migrações

Os scripts de migração devem ser armazenados no diretório padrão da aplicação.

```
backend/  
└─ src/  
    └─ main/  
        └─ resources/  
            └─ db/  
                └─ migration/  
                    ├── V1__Criacao_Inicial.sql  
                    ├── V2__Usuarios.sql  
                    ├── V3__Territorios.sql  
                    ├── V4__Publicacoes.sql  
                    └─ ...
```

Cada arquivo representa uma alteração incremental no esquema do banco.

## 5.10 Convenção de Nomenclatura

As migrações devem seguir uma convenção padronizada.

Exemplos:

| Arquivo                 | Finalidade              |
|-------------------------|-------------------------|
| V1__Criacao_Inicial.sql | Estrutura inicial       |
| V2__Usuarios.sql        | Tabela de usuários      |
| V3__Perfis.sql          | Perfis e permissões     |
| V4__Territorios.sql     | Cadastro de territórios |
| V5__Publicacoes.sql     | Cadastro de publicações |

A sequência numérica nunca deve ser alterada após a migração ser aplicada.

## 5.11 Execução das Migrações

Durante a inicialização da aplicação, o Flyway executará automaticamente todas as migrações pendentes.

Caso seja necessário validar a execução, recomenda-se verificar:

- histórico de migrações;
- tabela de controle do Flyway;
- logs da aplicação;
- estrutura final do banco.

Em condições normais, nenhuma intervenção manual será necessária.

## 5.12 Validação da Estrutura

Após a execução das migrações, recomenda-se validar:

- existência das tabelas;
- índices;
- chaves primárias;
- chaves estrangeiras;
- restrições;
- sequências;
- views (quando utilizadas).

Essa validação garante que a estrutura esperada foi criada corretamente.

## 5.13 Evolução do Banco de Dados

Sempre que houver necessidade de alterar o esquema, recomenda-se:

1. criar uma nova migração;
2. manter compatibilidade com versões anteriores, quando possível;
3. testar em ambiente de desenvolvimento;
4. validar em homologação;
5. aplicar em produção por meio do processo oficial de implantação.

Migrações já executadas não devem ser alteradas.



## 5.14 Backup Antes das Migrações

Antes da implantação de novas versões, recomenda-se:

- realizar backup completo do banco de dados;
- validar a integridade do backup;
- documentar a versão da aplicação;
- garantir a possibilidade de recuperação em caso de falha.

Essa prática reduz riscos durante atualizações.

## 5.15 Problemas Frequentes

| Problema                          | Possível Solução                                                      |
|-----------------------------------|-----------------------------------------------------------------------|
| Falha de conexão com o PostgreSQL | Verificar serviço, porta, usuário e senha                             |
| Flyway não executa migrações      | Confirmar localização dos scripts e configuração do Flyway            |
| Erro de permissão                 | Revisar privilégios do usuário da aplicação                           |
| Banco em versão diferente         | Validar o histórico de migrações antes de prosseguir                  |
| Estrutura inconsistente           | Comparar o banco com as migrações disponíveis e corrigir divergências |

## 5.16 Boas Práticas

Para garantir uma camada de persistência consistente, recomenda-se:

- utilizar um banco exclusivo para cada ambiente;
- manter as migrações sob controle de versão;
- nunca editar migrações já executadas;
- documentar alterações estruturais relevantes;
- realizar backups antes de atualizações;
- utilizar usuários com permissões mínimas;

- monitorar regularmente o desempenho do banco de dados.

## 5.17 Conclusão do Capítulo

Este capítulo apresentou os procedimentos necessários para configurar o PostgreSQL e o Flyway no GTP, abordando a criação do banco de dados, a configuração da conexão, a organização das migrações e as boas práticas para evolução do esquema de persistência.

Com essa configuração concluída, o banco de dados estará preparado para suportar as funcionalidades da aplicação de forma consistente, segura e versionada.

## 6 – Docker e Docker Compose

### 6.1 Objetivo

Este capítulo apresenta a utilização do **Docker** e do **Docker Compose** no **Gestor de Territórios e Publicações (GTP)**.

Serão abordados os procedimentos para criação, configuração e execução dos contêineres da aplicação, permitindo que todos os ambientes utilizem uma infraestrutura padronizada, reproduzível e de fácil manutenção.

O uso de contêineres simplifica o processo de instalação, reduz diferenças entre ambientes e facilita a implantação contínua.

### 6.2 Visão Geral

O GTP utiliza Docker para empacotar seus componentes e Docker Compose para orquestrar os serviços necessários ao funcionamento da plataforma.

A arquitetura é composta pelos seguintes elementos:

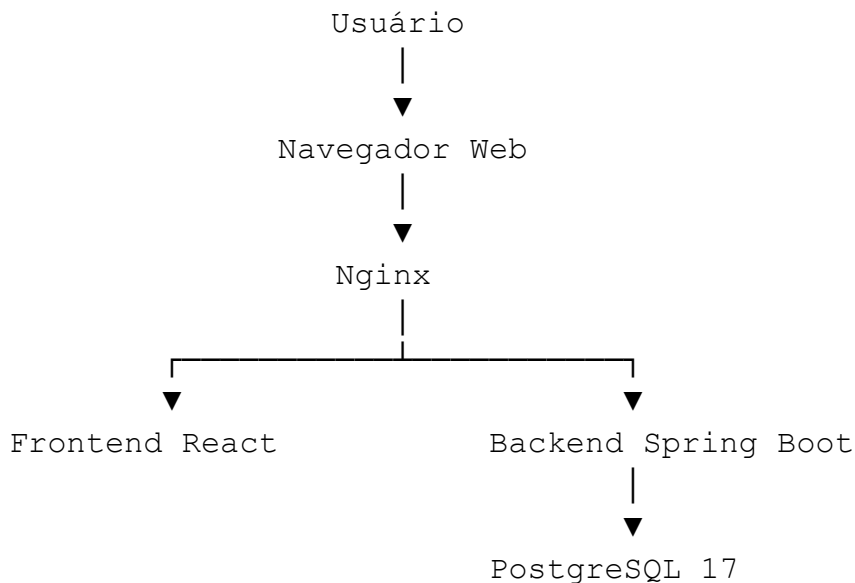
- Frontend React;
- Backend Spring Boot;
- PostgreSQL;
- Nginx (ambientes de produção);
- Volumes persistentes;

- Redes internas Docker.

Essa organização garante isolamento entre os serviços e facilita futuras expansões da infraestrutura.

## 6.3 Arquitetura dos Contêineres

A comunicação entre os componentes ocorre por meio da rede interna do Docker.



Cada componente é executado em um contêiner independente, promovendo desacoplamento e facilidade de manutenção.

## 6.4 Estrutura Recomendada

A organização dos arquivos relacionados ao Docker pode seguir a estrutura abaixo:

```
docker/
├── backend/
│   └── Dockerfile
├── frontend/
│   └── Dockerfile
├── nginx/
│   ├── Dockerfile
│   └── nginx.conf
└── postgres/
```

Na raiz do projeto:

```
docker-compose.yml
docker-compose.override.yml
.env
```

Essa separação facilita a manutenção e a evolução da infraestrutura.

## 6.5 Dockerfile do Backend

O contêiner do backend é responsável por executar a aplicação Spring Boot.

Recomenda-se utilizar:

- Java 21 LTS;
- build em múltiplos estágios (*multi-stage build*);
- imagem final enxuta;
- usuário não privilegiado;
- exposição apenas da porta necessária.

Essa abordagem reduz o tamanho da imagem e melhora a segurança.

## 6.6 Dockerfile do Frontend

O frontend pode ser executado de duas formas:

### Desenvolvimento

Utilizando o servidor do Vite para recarga automática (*Hot Module Replacement*).

### Produção

Gerando os arquivos estáticos e disponibilizando-os por meio do Nginx.

Essa separação otimiza o desempenho em produção e melhora a experiência durante o desenvolvimento.

## 6.7 Serviço PostgreSQL

O contêiner do banco de dados deve incluir:

- volume persistente;

- usuário dedicado da aplicação;
- senha protegida por variáveis de ambiente;
- banco inicial configurado automaticamente;
- codificação UTF-8.

A persistência dos dados deve ser realizada por meio de volumes Docker, evitando perda de informações durante recriações dos contêineres.

## 6.8 Docker Compose

O Docker Compose é responsável por iniciar todos os serviços necessários ao funcionamento do GTP.

A configuração deve contemplar:

| Serviço  | Finalidade               |
|----------|--------------------------|
| frontend | Interface React          |
| backend  | API Spring Boot          |
| postgres | Banco de Dados           |
| nginx    | Proxy reverso (produção) |

O Compose também define:

- redes;
- volumes;
- dependências entre serviços;
- variáveis de ambiente;
- políticas de reinicialização.

## 6.9 Variáveis de Ambiente

As configurações específicas de cada ambiente devem ser fornecidas por meio de um arquivo `.env`.

Exemplos de informações configuradas:

- nome do banco de dados;

- usuário;
- senha;
- porta da aplicação;
- perfil Spring ativo;
- URL da API;
- chave JWT;
- parâmetros de integração.

Recomenda-se que o arquivo `.env` não seja versionado quando contiver informações sensíveis.

## 6.10 Inicialização do Ambiente

Após configurar os arquivos necessários, o ambiente completo pode ser iniciado com:

```
docker compose up -d
```

Esse comando:

- cria a rede Docker;
- inicia os contêineres;
- monta os volumes;
- executa os serviços em segundo plano.

Na primeira execução, as imagens serão construídas automaticamente, caso ainda não existam.

## 6.11 Verificação dos Serviços

Após a inicialização, recomenda-se validar:

- status dos contêineres;
- disponibilidade do backend;
- acesso ao frontend;
- conexão com o PostgreSQL;

- execução das migrações Flyway;
- comunicação entre os serviços.

A inspeção dos logs é fundamental para identificar possíveis falhas durante a inicialização.

## 6.12 Persistência de Dados

Os dados do PostgreSQL devem ser armazenados em volumes persistentes.

Além do banco de dados, recomenda-se persistir:

- uploads de usuários;
- certificados digitais (quando utilizados);
- arquivos de configuração;
- logs relevantes.

Essa estratégia evita perda de dados em caso de atualização ou recriação dos contêineres.

## 6.13 Redes Docker

Os serviços do GTP devem se comunicar por meio de uma rede privada Docker.

Benefícios dessa abordagem:

- isolamento entre aplicações;
- comunicação interna segura;
- resolução automática de nomes dos serviços;
- facilidade de escalabilidade.

A exposição de portas deve ser limitada apenas aos serviços que necessitam acesso externo.

## 6.14 Atualização dos Contêineres

Sempre que houver novas versões da aplicação, recomenda-se:

1. atualizar o código-fonte;

2. reconstruir as imagens;
3. validar as migrações do banco;
4. reiniciar os serviços;
5. verificar os logs;
6. confirmar o funcionamento da aplicação.

Antes da atualização em produção, é recomendável realizar backup do banco de dados.

## 6.15 Problemas Frequentes

| Problema                                | Possível Solução                                                   |
|-----------------------------------------|--------------------------------------------------------------------|
| Contêiner não inicia                    | Verificar os logs e a configuração do Dockerfile                   |
| Porta em uso                            | Alterar a porta configurada ou encerrar o processo conflitante     |
| Backend não conecta ao banco            | Confirmar rede Docker, credenciais e disponibilidade do PostgreSQL |
| Volume não persiste dados               | Validar o mapeamento dos volumes no Docker Compose                 |
| Falha na construção da imagem           | Limpar o cache do Docker e reconstruir as imagens                  |
| Comunicação entre serviços não funciona | Verificar a configuração da rede e os nomes dos serviços           |

## 6.16 Boas Práticas

Para manter uma infraestrutura baseada em contêineres organizada e segura, recomenda-se:

- utilizar imagens oficiais sempre que possível;
- manter as imagens atualizadas;
- utilizar *multi-stage builds* para reduzir o tamanho das imagens;
- evitar executar contêineres como usuário root;



- utilizar variáveis de ambiente para configurações específicas;
- persistir dados críticos em volumes dedicados;
- monitorar regularmente o consumo de recursos dos contêineres;
- documentar alterações na infraestrutura.

## 6.17 Conclusão do Capítulo

Este capítulo apresentou a configuração do Docker e do Docker Compose no GTP, detalhando a organização dos contêineres, a persistência de dados, a orquestração dos serviços e as boas práticas para ambientes de desenvolvimento e produção.

Com essa configuração, a plataforma passa a contar com um ambiente padronizado, reproduzível e de fácil implantação, reduzindo diferenças entre ambientes e facilitando a manutenção da infraestrutura.

## 7 – Variáveis de Ambiente e Segurança

### 7.1 Objetivo

Este capítulo apresenta as diretrizes para configuração das variáveis de ambiente e das políticas de segurança utilizadas pelo **Gestor de Territórios e Publicações (GTP)**.

O objetivo é garantir que informações sensíveis sejam protegidas, que cada ambiente possua configurações próprias e que a aplicação siga boas práticas de segurança durante o desenvolvimento, homologação e produção.

### 7.2 Visão Geral

As variáveis de ambiente permitem separar a configuração da aplicação do seu código-fonte.

Essa abordagem oferece diversos benefícios:

- maior segurança das credenciais;
- facilidade para implantação em diferentes ambientes;
- redução de alterações no código;

- padronização da configuração;
- integração com Docker, Kubernetes e plataformas de nuvem.

O GTP utiliza variáveis de ambiente tanto no backend quanto no frontend, respeitando as necessidades de cada componente.

## 7.3 Organização por Ambiente

Cada ambiente deve possuir sua própria configuração.

| Ambiente              | Finalidade                       |
|-----------------------|----------------------------------|
| Desenvolvimento (dev) | Desenvolvimento local            |
| Testes (test)         | Execução de testes automatizados |
| Homologação (hom)     | Validação antes da produção      |
| Produção (prod)       | Utilização pelos usuários finais |

As configurações devem ser independentes para evitar impactos entre ambientes.

## 7.4 Variáveis do Backend

O backend utiliza variáveis para parametrizar componentes essenciais da aplicação.

Entre elas:

- perfil ativo do Spring Boot;
- URL do banco de dados;
- usuário do PostgreSQL;
- senha do banco;
- chave secreta do JWT;
- tempo de expiração dos tokens;
- configuração de CORS;
- parâmetros de envio de e-mails (SMTP);
- configurações de logs;

- integrações com serviços externos.

Essas variáveis devem ser definidas no ambiente de execução e nunca incorporadas ao código-fonte.

## 7.5 Variáveis do Frontend

O frontend utiliza variáveis para configurar aspectos que podem variar entre ambientes.

Exemplos:

- URL da API REST;
- nome da aplicação;
- ambiente de execução;
- funcionalidades opcionais (*feature flags*);
- informações públicas de integração.

No Vite, todas as variáveis acessíveis ao frontend devem utilizar o prefixo `VITE_`.

Como o código do frontend é executado no navegador, nenhuma informação sensível deve ser exposta por meio dessas variáveis.

## 7.6 Arquivos de Configuração

Durante o desenvolvimento, é comum utilizar arquivos específicos para armazenar variáveis de ambiente.

Exemplo de organização:

```
backend/
├── .env
├── .env.dev
├── .env.test
└── .env.prod

frontend/
├── .env
├── .env.development
├── .env.production
└── .env.local
```

Cada arquivo deve conter apenas as configurações necessárias para o respectivo ambiente.

## 7.7 Proteção de Credenciais

As credenciais da aplicação devem ser tratadas como informações confidenciais.

Boas práticas incluem:

- utilizar senhas fortes;
- evitar compartilhamento de credenciais;
- restringir o acesso aos arquivos de configuração;
- armazenar segredos em serviços especializados (*Secrets Managers*), quando disponíveis;
- realizar rotação periódica de senhas e chaves.

Essas medidas reduzem o risco de comprometimento da infraestrutura.

## 7.8 Gerenciamento de Chaves e Tokens

O GTP utiliza mecanismos de autenticação baseados em tokens.

Os administradores devem garantir:

- geração de chaves criptográficas seguras;
- armazenamento protegido das chaves;
- renovação periódica dos segredos;
- definição de tempo adequado para expiração dos tokens;
- revogação de credenciais comprometidas.

Esses cuidados fortalecem a segurança da autenticação.

## 7.9 Configuração de HTTPS

Em ambiente de produção, recomenda-se que toda a comunicação ocorra por meio de HTTPS.

Benefícios:

- criptografia dos dados em trânsito;
- proteção contra interceptação de informações;
- autenticação do servidor;
- aumento da confiança dos usuários.

A utilização de certificados digitais válidos é obrigatória para ambientes públicos.

## 7.10 Controle de Acesso às Configurações

O acesso às configurações da aplicação deve ser restrito.

Recomenda-se:

- conceder acesso apenas a administradores autorizados;
- registrar alterações críticas;
- manter histórico das modificações;
- revisar permissões periodicamente.

A rastreabilidade das alterações facilita auditorias e investigações.

## 7.11 Versionamento de Arquivos

Arquivos contendo informações sensíveis não devem ser armazenados em repositórios públicos.

Recomenda-se:

- adicionar arquivos de configuração sensíveis ao `.gitignore`;
- manter apenas arquivos de exemplo (como `.env.example`) sob controle de versão;
- documentar todas as variáveis obrigatórias.

Essa abordagem permite que novos ambientes sejam configurados sem expor credenciais reais.

## 7.12 Boas Práticas de Segurança

Para manter um ambiente seguro, recomenda-se:

- utilizar autenticação multifator (MFA) para acessos administrativos;
- revisar periodicamente as credenciais;
- limitar permissões de usuários e serviços;
- criptografar informações sensíveis;
- manter dependências atualizadas;
- monitorar tentativas de acesso indevido;
- revisar periodicamente as configurações de segurança.

Essas práticas devem fazer parte da rotina de administração do sistema.

## 7.13 Problemas Frequentes

| Problema                                      | Possível Solução                                                                                                |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Variável de ambiente não reconhecida          | Confirmar o nome da variável e reiniciar a aplicação após a alteração                                           |
| Erro de autenticação com o banco              | Revisar usuário, senha e URL de conexão                                                                         |
| Chave JWT inválida                            | Gerar uma nova chave e atualizar a configuração do ambiente                                                     |
| Configuração diferente entre ambientes        | Comparar os arquivos de configuração e padronizar os valores necessários                                        |
| Informações sensíveis expostas no repositório | Remover imediatamente as credenciais, rotacionar os segredos comprometidos e revisar o histórico do repositório |

## 7.14 Checklist de Segurança

Antes de disponibilizar o ambiente para uso, recomenda-se verificar:

- credenciais protegidas;
- variáveis de ambiente configuradas corretamente;
- arquivos sensíveis fora do controle de versão;

- HTTPS habilitado em produção;
- chaves criptográficas atualizadas;
- backups das configurações críticas;
- revisão das permissões administrativas.

Esse checklist contribui para reduzir riscos de segurança durante a implantação.

## 7.15 Relação com os Próximos Capítulos

As configurações de ambiente e segurança apresentadas neste capítulo servem como base para a implantação em produção.

No próximo capítulo serão abordados:

- preparação do ambiente produtivo;
- configuração do servidor;
- publicação da aplicação;
- proxy reverso;
- certificados digitais;
- monitoramento pós-implantação;
- validação da disponibilidade do sistema.

## 7.16 Conclusão do Capítulo

Este capítulo apresentou as diretrizes para configuração das variáveis de ambiente e para a proteção das informações sensíveis utilizadas pelo GTP.

A correta separação das configurações por ambiente, o gerenciamento adequado de credenciais e a adoção de boas práticas de segurança contribuem para uma implantação mais segura, organizada e alinhada às recomendações atuais de desenvolvimento e operação de aplicações.

## 8 – Configuração para Produção

### 8.1 Objetivo

Este capítulo apresenta as diretrizes para implantação do **Gestor de Territórios e Publicações (GTP)** em ambiente de produção.

Serão abordadas as etapas de preparação da infraestrutura, configuração dos serviços, publicação da aplicação, validação da implantação e monitoramento inicial, garantindo que a plataforma opere de forma segura, estável e com alta disponibilidade.

### 8.2 Visão Geral

O ambiente de produção é destinado ao uso pelos usuários finais e deve seguir requisitos mais rigorosos de segurança, desempenho e disponibilidade.

A infraestrutura recomendada é composta por:

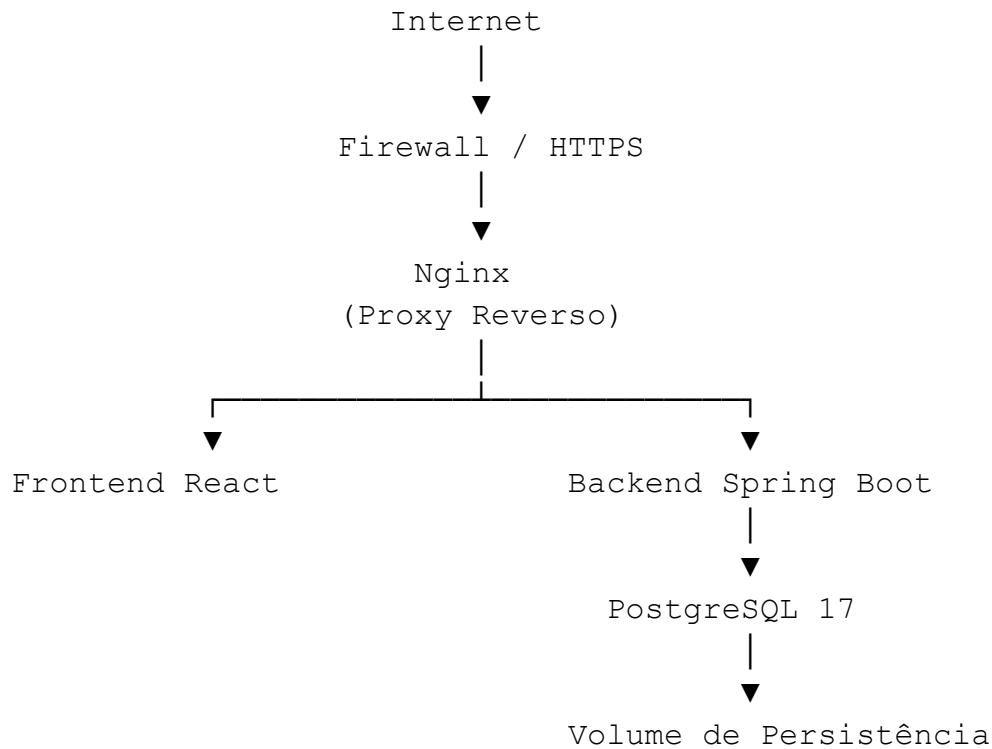
- servidor Linux;
- Docker e Docker Compose;
- PostgreSQL;
- Backend Spring Boot;
- Frontend React;
- Nginx como proxy reverso;
- HTTPS com certificado digital;
- sistema de monitoramento;
- rotina de backup.

Todos os componentes devem ser configurados para operar de forma integrada.

### 8.3 Arquitetura do Ambiente de Produção

A arquitetura recomendada é apresentada a seguir.





Essa arquitetura favorece a segurança, a escalabilidade e a separação de responsabilidades entre os componentes.

## 8.4 Preparação do Servidor

Antes da implantação, recomenda-se verificar:

- sistema operacional atualizado;
- acesso administrativo controlado;
- sincronização de data e hora;
- firewall configurado;
- Docker instalado;
- Docker Compose disponível;
- espaço em disco suficiente;
- memória e processamento compatíveis com a carga prevista.

Também é recomendável restringir o acesso remoto apenas aos administradores autorizados.

## 8.5 Configuração do Backend

O backend deve ser configurado para utilizar o perfil de produção.

Recomenda-se:

- habilitar o perfil prod;
- utilizar variáveis de ambiente para todas as configurações sensíveis;
- desabilitar recursos exclusivos de desenvolvimento;
- configurar níveis adequados de log;
- validar a conexão com o banco de dados;
- confirmar a execução automática das migrações Flyway.

Essas medidas contribuem para maior segurança e estabilidade.

## 8.6 Configuração do Frontend

O frontend deve ser compilado em modo de produção.

O processo de publicação deve contemplar:

- geração dos arquivos otimizados;
- disponibilização dos arquivos estáticos;
- configuração do servidor web;
- integração com a API do backend;
- validação das rotas da aplicação.

A versão publicada deve corresponder exatamente à versão do backend implantado.

## 8.7 Configuração do Nginx

O Nginx atua como proxy reverso da aplicação.

Suas principais responsabilidades incluem:

- distribuição das requisições;
- entrega dos arquivos estáticos;

- redirecionamento para HTTPS;
- compressão de conteúdo;
- cache de recursos estáticos;
- encaminhamento das requisições para o backend.

A configuração deve ser revisada sempre que houver alterações significativas na infraestrutura.

## 8.8 Configuração do HTTPS

Toda comunicação com o GTP deve ocorrer utilizando HTTPS.

A configuração deve contemplar:

- certificado digital válido;
- renovação periódica do certificado;
- desativação de protocolos inseguros;
- utilização de algoritmos criptográficos modernos;
- redirecionamento automático de HTTP para HTTPS.

Essas práticas garantem a confidencialidade e a integridade das informações trafegadas.

## 8.9 Configuração do Banco de Dados

O PostgreSQL em produção deve ser configurado com foco em segurança e desempenho.

Recomenda-se:

- utilizar usuário exclusivo da aplicação;
- restringir conexões externas;
- habilitar backups automáticos;
- monitorar utilização de recursos;
- revisar índices periodicamente;
- controlar permissões administrativas.

As migrações do Flyway devem ser executadas antes da disponibilização da aplicação aos usuários.

## 8.10 Variáveis de Ambiente

Todas as configurações específicas de produção devem ser fornecidas por meio de variáveis de ambiente.

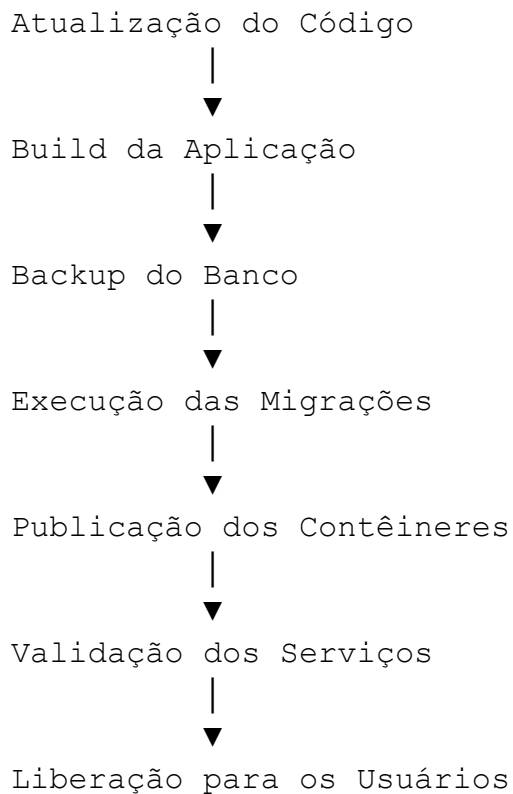
Exemplos:

- perfil Spring ativo;
- parâmetros do banco de dados;
- chaves JWT;
- configurações SMTP;
- URLs dos serviços;
- parâmetros de monitoramento.

Nenhuma informação sensível deve ser armazenada diretamente no código-fonte.

## 8.11 Publicação da Aplicação

Fluxo recomendado para implantação:



Esse processo reduz riscos durante atualizações.

## 8.12 Validação Pós-Implantação

Após a publicação, recomenda-se verificar:

- disponibilidade da aplicação;
- funcionamento do login;
- comunicação entre frontend e backend;
- acesso ao banco de dados;
- execução das funcionalidades principais;
- integridade dos dados;
- registros de log.

Qualquer inconsistência deve ser tratada antes da liberação definitiva.

## 8.13 Monitoramento Inicial

Nas primeiras horas após a implantação, é importante acompanhar:

- utilização de CPU;
- consumo de memória;
- utilização do disco;
- conexões com o banco de dados;
- tempo médio de resposta;
- erros registrados nos logs;
- disponibilidade da aplicação.

Esse monitoramento auxilia na identificação precoce de problemas.

## 8.14 Plano de Rollback

Toda implantação deve possuir um plano de reversão.

Caso ocorra alguma falha crítica, recomenda-se:

1. interromper a implantação;
2. restaurar a versão anterior da aplicação;
3. restaurar o banco de dados, quando necessário;
4. validar o funcionamento da versão restaurada;
5. registrar o incidente e sua causa.

O plano de rollback deve ser documentado e testado periodicamente.

## 8.15 Checklist de Implantação

Antes de disponibilizar a aplicação em produção, confirme:

- servidor atualizado;
- Docker funcionando corretamente;
- banco de dados disponível;
- migrações Flyway executadas;
- backend operacional;
- frontend publicado;
- HTTPS configurado;
- backup realizado;
- monitoramento habilitado;
- logs revisados.

Esse checklist reduz a probabilidade de falhas durante a implantação.

## 8.16 Problemas Frequentes

| Problema                                 | Possível Solução                             |
|------------------------------------------|----------------------------------------------|
| Aplicação indisponível após implantação  | Verificar os logs e o status dos contêineres |
| Erro de conexão entre frontend e backend | Revisar URLs, proxy reverso e regras de CORS |

| <b>Problema</b>            | <b>Possível Solução</b>                                                   |
|----------------------------|---------------------------------------------------------------------------|
| Certificado HTTPS inválido | Confirmar a instalação e a validade do certificado                        |
| Migrações não executadas   | Validar a configuração do Flyway e o acesso ao banco de dados             |
| Lentidão da aplicação      | Monitorar recursos do servidor e revisar a configuração da infraestrutura |

## 8.17 Boas Práticas

Para um ambiente de produção seguro e estável, recomenda-se:

- automatizar o processo de implantação;
- utilizar integração e entrega contínuas (CI/CD);
- manter backups atualizados;
- monitorar continuamente os serviços;
- revisar periodicamente os certificados digitais;
- restringir acessos administrativos;
- manter todas as dependências atualizadas;
- documentar cada implantação realizada.

## 8.18 Conclusão do Capítulo

Este capítulo apresentou os procedimentos para configuração do ambiente de produção do GTP, abrangendo a preparação da infraestrutura, a publicação da aplicação, a configuração dos serviços essenciais e a validação pós-implantação.

A adoção dessas práticas garante um ambiente mais seguro, estável e preparado para atender aos usuários finais com alta disponibilidade e confiabilidade.

## 9 – Solução de Problemas

### 9.1 Objetivo

Este capítulo apresenta os principais procedimentos para identificação, diagnóstico e resolução de problemas relacionados ao **Gestor de Territórios e Publicações (GTP)**.

O objetivo é fornecer um guia prático para administradores, desenvolvedores e equipes de suporte, reduzindo o tempo de indisponibilidade da plataforma e facilitando a recuperação de falhas.

### 9.2 Visão Geral

Problemas podem ocorrer em qualquer camada da aplicação.

As principais áreas que devem ser verificadas são:

- infraestrutura;
- Docker;
- Backend Spring Boot;
- Frontend React;
- PostgreSQL;
- Flyway;
- rede;
- autenticação;
- integração entre serviços.

A identificação correta da origem da falha reduz significativamente o tempo necessário para sua resolução.

### 9.3 Processo de Diagnóstico

Sempre que um problema for identificado, recomenda-se seguir um fluxo padronizado.



Identificação do Problema



Coleta de Evidências



Análise dos Logs



Identificação da Causa



Aplicação da Correção



Validação



Documentação da Solução

Esse procedimento contribui para uma resolução consistente e para a construção de uma base de conhecimento.

## 9.4 Problemas no Backend

### Sintoma

A aplicação Spring Boot não inicia.

### Possíveis causas

- erro de compilação;
- dependências incompatíveis;
- configuração incorreta do application.yml;
- porta ocupada;
- erro de conexão com o banco de dados.

### Procedimentos recomendados

- verificar os logs de inicialização;
- confirmar a versão do Java;
- revisar as dependências Maven;

- validar as variáveis de ambiente;
- confirmar se o PostgreSQL está disponível.

## 9.5 Problemas no Frontend

### Sintoma

A interface não é carregada ou apresenta tela em branco.

### Possíveis causas

- erro de compilação;
- dependências ausentes;
- URL incorreta da API;
- falhas de JavaScript;
- recursos estáticos não encontrados.

### Procedimentos recomendados

- verificar o console do navegador;
- confirmar a configuração das variáveis de ambiente;
- validar se o backend está em execução;
- revisar o processo de build.

## 9.6 Problemas de Comunicação entre Frontend e Backend

### Sintoma

A interface não consegue acessar a API.

### Possíveis causas

- backend indisponível;
- URL incorreta;
- erro de CORS;
- proxy reverso mal configurado;

- falhas de autenticação.

### **Procedimentos recomendados**

- confirmar o endereço da API;
- verificar as configurações de CORS;
- revisar a configuração do Nginx;
- validar os logs do backend.

## **9.7 Problemas no PostgreSQL**

### **Sintoma**

A aplicação não consegue acessar o banco de dados.

### **Possíveis causas**

- serviço parado;
- credenciais incorretas;
- banco inexistente;
- porta bloqueada;
- limite de conexões atingido.

### **Procedimentos recomendados**

- verificar o status do serviço PostgreSQL;
- confirmar usuário e senha;
- validar a URL de conexão;
- revisar os logs do banco de dados.

## **9.8 Problemas com Flyway**

### **Sintoma**

As migrações não são executadas.

### **Possíveis causas**

- scripts fora do diretório esperado;

- erro de sintaxe SQL;
- migração duplicada;
- histórico inconsistente.

### **Procedimentos recomendados**

- revisar os scripts de migração;
- verificar o histórico do Flyway;
- validar a sequência das versões;
- corrigir inconsistências antes de prosseguir.

## **9.9 Problemas com Docker**

### **Sintoma**

Os contêineres não iniciam corretamente.

### **Possíveis causas**

- erro no Dockerfile;
- imagem corrompida;
- conflito de portas;
- configuração incorreta do Docker Compose.

### **Procedimentos recomendados**

- consultar os logs dos contêineres;
- reconstruir as imagens;
- verificar volumes e redes;
- confirmar as variáveis de ambiente.

## **9.10 Problemas de Autenticação**

### **Sintoma**

O login é recusado.

### **Possíveis causas**

- credenciais inválidas;
- token expirado;
- chave JWT incorreta;
- permissões insuficientes.

### **Procedimentos recomendados**

- validar usuário e senha;
- revisar a configuração do JWT;
- verificar o perfil do usuário;
- analisar os logs de autenticação.

## **9.11 Problemas de Desempenho**

### **Sintoma**

A aplicação apresenta lentidão.

### **Possíveis causas**

- consultas SQL ineficientes;
- uso excessivo de memória;
- carga elevada no servidor;
- índices ausentes;
- gargalos na infraestrutura.

### **Procedimentos recomendados**

- monitorar CPU e memória;
- revisar consultas ao banco;
- analisar os tempos de resposta;
- otimizar índices e consultas.

## 9.12 Problemas de HTTPS

### Sintoma

O navegador informa conexão insegura.

### Possíveis causas

- certificado expirado;
- domínio incorreto;
- configuração inadequada do Nginx;
- cadeia de certificados incompleta.

### Procedimentos recomendados

- verificar a validade do certificado;
- revisar a configuração SSL/TLS;
- confirmar o domínio configurado;
- reiniciar o servidor após alterações.

## 9.13 Análise de Logs

Os logs são a principal fonte de informações para diagnóstico.

Recomenda-se analisar:

- logs do backend;
- logs do frontend (console do navegador);
- logs do PostgreSQL;
- logs do Docker;
- logs do Nginx;
- registros do sistema operacional.

Sempre que possível, correlacione eventos entre diferentes componentes para identificar a causa raiz do problema.

## 9.14 Checklist de Diagnóstico

Antes de acionar a equipe responsável, verifique:

- aplicação em execução;
- disponibilidade do banco de dados;
- funcionamento da rede;
- status dos contêineres;
- validade das credenciais;
- configuração das variáveis de ambiente;
- existência de mensagens de erro nos logs;
- disponibilidade de espaço em disco;
- consumo de CPU e memória.

## 9.15 Tabela de Problemas Comuns

| Problema                | Possível Solução                                           |
|-------------------------|------------------------------------------------------------|
| Backend não inicia      | Revisar logs, Java, Maven e configurações do Spring Boot   |
| Frontend não carrega    | Executar novamente o build e validar a configuração da API |
| PostgreSQL indisponível | Reiniciar o serviço e validar as credenciais               |
| Docker não sobe         | Reconstruir as imagens e revisar o Docker Compose          |
| Erro de CORS            | Ajustar a configuração de CORS no backend                  |
| Login falha             | Verificar autenticação, permissões e token JWT             |
| HTTPS inválido          | Renovar ou reinstalar o certificado digital                |
| Migração Flyway falha   | Corrigir a migração e validar o histórico do Flyway        |

## 9.16 Registro de Incidentes

Cada incidente deve ser documentado contendo, no mínimo:

- data e horário;
- ambiente afetado;
- descrição do problema;
- impacto para os usuários;
- causa identificada;
- ações executadas;
- responsável pela correção;
- data da resolução;
- ações preventivas para evitar recorrência.

Esse registro contribui para auditorias e para a melhoria contínua da operação.

## 9.17 Boas Práticas

Para reduzir a ocorrência de falhas, recomenda-se:

- manter todos os componentes atualizados;
- monitorar continuamente os serviços;
- revisar regularmente os logs;
- executar backups periódicos;
- validar as implantações em homologação antes da produção;
- documentar alterações de infraestrutura;
- automatizar verificações de integridade sempre que possível.

## 9.18 Conclusão do Capítulo

Este capítulo apresentou os procedimentos para diagnóstico e resolução dos problemas mais comuns encontrados durante a instalação, configuração e operação do GTP.



A utilização de um processo estruturado de análise, aliada à documentação dos incidentes e ao monitoramento contínuo da infraestrutura, contribui para reduzir o tempo de indisponibilidade e aumentar a confiabilidade da plataforma.

## 10 – Conclusão

### 10.1 Objetivo

Este capítulo conclui o **Guia de Instalação e Configuração do Gestor de Territórios e Publicações (GTP)**, consolidando as diretrizes apresentadas ao longo do documento e reforçando a importância da padronização dos ambientes para garantir uma implantação segura, estável e reproduzível.

Também são apresentadas recomendações para a manutenção contínua da infraestrutura e para a evolução do processo de instalação da plataforma.

### 10.2 Síntese do Guia

Ao longo deste documento foram apresentados todos os procedimentos necessários para preparar um ambiente completo de execução do GTP.

Foram abordados os seguintes temas:

- preparação do ambiente de desenvolvimento;
- configuração do backend;
- configuração do frontend;
- instalação e configuração do PostgreSQL;
- gerenciamento das migrações com Flyway;
- utilização de Docker e Docker Compose;
- configuração das variáveis de ambiente;
- implantação em ambiente de produção;
- diagnóstico e solução de problemas.

Esses procedimentos estabelecem uma base consistente para instalação e manutenção da plataforma em diferentes ambientes.

## 10.3 Padronização dos Ambientes

A utilização de ambientes padronizados reduz diferenças entre desenvolvimento, homologação e produção.

Entre os principais benefícios estão:

- maior previsibilidade das implantações;
- redução de erros de configuração;
- facilidade para integração de novos desenvolvedores;
- simplificação do processo de suporte;
- maior estabilidade operacional.

A padronização deve ser mantida durante todo o ciclo de vida do projeto.

## 10.4 Automação da Infraestrutura

Sempre que possível, recomenda-se automatizar os processos de instalação e implantação.

Exemplos:

- execução automática das migrações Flyway;
- construção automatizada das imagens Docker;
- pipelines de Integração Contínua (CI);
- pipelines de Entrega Contínua (CD);
- validação automática de qualidade;
- implantação automatizada em homologação e produção.

A automação reduz falhas humanas e aumenta a confiabilidade das entregas.

## 10.5 Segurança da Configuração

As configurações da aplicação devem seguir princípios de segurança durante todo o ciclo de vida.

Recomenda-se:

- utilizar variáveis de ambiente para informações sensíveis;

- proteger credenciais e chaves criptográficas;
- restringir acessos administrativos;
- revisar permissões periodicamente;
- manter dependências atualizadas;
- utilizar conexões seguras (HTTPS/TLS).

Essas práticas fortalecem a segurança da infraestrutura e protegem os dados da aplicação.

## 10.6 Manutenção Contínua

Após a implantação, a infraestrutura deve ser acompanhada continuamente.

As principais atividades incluem:

- atualização das dependências;
- monitoramento da disponibilidade;
- revisão de logs;
- manutenção preventiva;
- verificação do banco de dados;
- atualização dos certificados digitais;
- testes periódicos de backup e recuperação.

Essas ações contribuem para manter a plataforma estável e segura.

## 10.7 Evolução da Infraestrutura

O GTP foi projetado para permitir crescimento gradual da infraestrutura.

Entre as evoluções previstas estão:

- implantação em ambientes de nuvem pública ou privada;
- utilização de orquestradores de contêineres, como Kubernetes;
- balanceamento de carga entre múltiplas instâncias;
- escalabilidade horizontal dos serviços;
- monitoramento avançado com ferramentas especializadas;

- gerenciamento centralizado de logs;
- integração com serviços de armazenamento distribuído.

Essas melhorias poderão ser incorporadas conforme o aumento da demanda e a evolução do projeto.

## 10.8 Integração com os Demais Documentos

Este guia complementa a documentação técnica e operacional do GTP.

Os principais documentos relacionados são:

| Documento                                | Finalidade                             |
|------------------------------------------|----------------------------------------|
| Documento 05 – Arquitetura Geral         | Visão arquitetural da solução          |
| Documento 06 – Arquitetura Backend       | Organização técnica do backend         |
| Documento 07 – Arquitetura Frontend      | Organização técnica do frontend        |
| Documento 09 – API REST                  | Interfaces de comunicação              |
| Documento 11 – Banco de Dados PostgreSQL | Modelagem e persistência               |
| Documento 13 – Estratégia de Testes      | Garantia da qualidade                  |
| Documento 14 – Deploy e Infraestrutura   | Arquitetura de implantação             |
| Documento 15 – Manual do Desenvolvedor   | Desenvolvimento e manutenção do código |
| Documento 16 – Manual do Usuário         | Utilização da plataforma               |
| Documento 17 – Manual do Administrador   | Administração e governança do sistema  |

Esses documentos, em conjunto, oferecem uma visão completa da arquitetura, desenvolvimento, operação e manutenção do GTP.

## 10.9 Recomendações para Futuras Versões

Para aperfeiçoar continuamente o processo de instalação e configuração, recomenda-se avaliar a adoção das seguintes iniciativas:

- scripts automatizados de instalação;
- infraestrutura como código (*Infrastructure as Code – IaC*);
- gerenciamento de segredos com ferramentas especializadas;
- implantação automatizada em múltiplos ambientes;
- testes automatizados de infraestrutura;
- provisionamento em provedores de nuvem;
- monitoramento centralizado;
- políticas avançadas de observabilidade.

Essas evoluções contribuirão para tornar a infraestrutura ainda mais robusta e preparada para ambientes de maior porte.

## 10.10 Considerações Finais

A correta instalação e configuração do **Gestor de Territórios e Publicações (GTP)** representa um dos pilares para o sucesso da plataforma.

Ao seguir as orientações apresentadas neste guia, administradores e desenvolvedores terão condições de implantar a aplicação de forma padronizada, reproduzível e segura, reduzindo riscos operacionais e facilitando futuras manutenções.

A combinação de boas práticas de desenvolvimento, automação, segurança e monitoramento contribui para uma infraestrutura confiável, preparada para acompanhar a evolução contínua do projeto.

## Encerramento do Documento

Com este capítulo, conclui-se o **Documento 18 – Guia de Instalação e Configuração**.

Este documento deve servir como referência para todas as atividades relacionadas à preparação de ambientes, instalação da aplicação, configuração da infraestrutura e implantação do GTP.

Sempre que houver alterações significativas na arquitetura, na infraestrutura ou nas ferramentas utilizadas, recomenda-se revisar e atualizar este guia para manter sua aderência à realidade do projeto.